CynipsNet v1.0 on the STM32N6570-DK: reverse engineering and improvement plan

Target: fotonik-sk/stm32n6-speech-recognition,
release v0.1.0. Question asked: can the model be reverse engineered, and can we train it further
to improve accuracy on the DK board's NPU?

Short answer: yes to both. The network architecture has been fully recovered from the compiled NPU program. The weights are plain int8 values in the flash image, not encrypted. The
audio front-end is fully specified in the C source. That is everything needed to either retrain the model from scratch or fine-tune the shipped weights.

1. What the repository ships

File	Content
`network_atonbuf.xSPI2.bin` (7.3 MB)	int8 weights and requantization vectors, flashed to NOR at `0x71500000`
`libcynipsnet.a`	compiled ST Edge AI output: the per-epoch NPU register programs. Symbols renamed to `w0000..w0229`
`converter_log.txt`	ST Edge AI Core 4.0 report: source was `epoch_04_step_00021311_v9.tflite` (not shipped), 7,199,645 params, 2.58 GMAC per run, 268 epochs, all on hardware
`asr_model_params.h`, `mel/`, `norm/`	the exact front-end and normalization in C

There is no ONNX, TFLite or training code. The author says they kept the generated C sources private.

2. Reverse engineering method

Renaming the symbols hides the layer names, but not the hardware configuration. Every layer must
program the NPU through `LL_Convacc_Init`, `LL_Arithacc_Init`, `LL_Streng_TensorInit` and
`LL_Switch_Init`, and the arguments are `static const` structs in `.rodata`.

1. `tools/gen_probe.py` compiles probe structs with the same ARM GCC and ABI (`-fshort-enums`,
   matching `Tag_ABI_enum_size: small`). This recovers the exact bit layout of every init struct.
2. `tools/decode_npu.py` parses the object (ELF and relocations) and resolves every call site's
   arguments. It walks the 268-entry epoch table in execution order and decodes each conv, arithmetic and stream-engine program, plus the stream-switch wiring between units.
3. Output: `out/epochs.json` (complete), `out/summary.txt` (per epoch), `out/compact.txt` (one line per epoch).

Reproduce it with `bash tools/run_decode.sh`. It needs only STM32CubeIDE's toolchain and Python.
REA (morluto/rea) was not needed. It drives
Ghidra, IDA or Hopper to decompile code, but this network is defined by data tables rather than
logic, and the GNU toolchain plus Python decoded those directly.

3. Recovered architecture: QuartzNet "6x5"

CynipsNet v1.0 is **QuartzNet 5x5 with the K=39 block doubled**. It is a QuartzNet-style CTC model made of
time-channel separable 1D convolutions, with BatchNorm folded into the pointwise convolutions.
PyTorch definition: `cynipsnet_arch.py`.

Block	Channels	Depthwise kernel	Sub-blocks	NPU epochs
C1	64 → 256	33, stride 2	1	1–4
B1	256	33	5 + residual 1x1	5–35
B2	256	39	5 + residual	36–72

B3	256	39	5 + residual	73–109
B4	256 → 512	51	5 + residual	110–146
B5	512	63	5 + residual	147–193
B6	512	75	5 + residual	194–244
C2	512	87, dilation 2	1	245–264
C3	512 → 1024	1	1	265
C4	1024 → 29	1	1	266

Evidence:

- **The parameter count matches exactly.** Running `python cynipsnet_arch.py` prints 7,199,645 as
  deployed, identical to the converter log. That count includes depthwise biases and C2's dilation stored as 173 taps. The trainable count is 7,155,613.
- **The dilation is confirmed in the weights.** In every C2 kernel chunk, all odd taps are exactly
  zero. In the B1 chunks, about 8 % of the values are zero.
- **The weight blob is accounted for.** About 99 % of the 7,315,128-byte blob is covered by decoded
  stream-engine weight reads and arithmetic-unit vectors.
- **The I/O shapes match.** Input 600×64, output 300×29 (stride 2).
- **The front-end matches NeMo's `AudioToMelSpectrogramPreprocessor` defaults.** These are 64 Slaney
  mels, 20 ms symmetric Hann window, 10 ms hop, n_fft 512, pre-emphasis 0.97, `log(x + 2^-24)` and
  per-feature normalization. The model was almost certainly trained with NVIDIA NeMo.
- **The labels are QuartzNet's English character set.** The only difference is that the blank is at index 0 instead of last.

Key systems fact: **the receptive field is about 33 s, but the window is 6 s.** Each window is effectively processed as an isolated, zero-padded clip. The firmware then commits only the middle
0.5 s of each window (420 frames of left context, 130 of right). So **each frame is computed**

about
12 times: 2.58 GMAC every 0.5 s, about 220 ms per run.

4. What still has to be decoded to fine-tune the shipped weights

- Map each layer's weight bytes to tensor order. The flash offsets are already in `epochs.json`. The
  depthwise layout is `[channel][tap]` per kernel chunk. The pointwise "deep mode" layout still has
  to be confirmed.
- Decode the requantization: the per-channel `A_vector` multipliers, `C_vector` biases and shifts
  of the arithmetic units, and the residual-add scalars. Then build a float-equivalent PyTorch model.
- Validate on a PC by running LibriSpeech dev-clean through the exact front-end. The target is
  about 5.79 % WER, the figure published for the quantized model. Optionally, compare the logits against the board's output over UART.

5. Improvement plan (ordered by value per effort)

A. Decoder and language model on the CPU (no retraining)

The firmware takes the argmax of int8 logits at every step, with no lexicon and no search.

- Add CTC prefix beam search on the Cortex-M55: 29 classes, 300 steps per window.
- Add a vocabulary or word n-gram for the target domain. For voice commands, a domain grammar,
  hotword list or contextual biasing usually gives the largest single accuracy gain. The author
  also lists a language model as the next step.

B. Train our own model with the recovered recipe (commercially clean)

- Use the same architecture or a larger one, in NeMo with the identical front-end and 29 labels.
  Export to TFLite int8 (per-channel), convert with **ST Edge AI 4.0 (installed at**

`C:\ST\STEdgeAI\4.0`), and drop the result into `models/`. Keep the I/O at 600×64 → 300×29
and update `network_shape.h` scales; then the firmware needs no other changes.

- Data is what makes it good on *our* board. The author's 248 h of board-microphone recordings are
  not released, so we have to build our own:
  - a replay rig: play a corpus through a speaker at 0.5–2.5 m and record through the DK microphone;
  - augmentation: the board microphone's measured frequency response, room impulse responses and noise;
  - accent coverage: Common Voice English with accent metadata, plus our own speakers. Svarah (9.6 h, 117 Indian speakers; CC BY 4.0 per the
    AIKosh listing) is a good
    held-out test set.
- Starting points: NVIDIA's pretrained
  QuartzNet 15x5
  uses the same front-end and character set and was trained on about 7,000 h. It can serve as a
  distillation teacher or as initialization. Its license is the NGC Terms of Use, so check commercial terms before shipping a derivative.
- No local GPU was found, so training needs a cloud or server GPU.

C. Fine-tune the shipped CynipsNet weights (fastest route to beating it)

- Finish section 4, then run quantization-aware fine-tuning on accent and board data. This keeps
  the author's board-microphone training.
- **License:** the weights are CC BY-NC-SA 4.0. Any fine-tuned derivative is non-commercial and
  ShareAlike. A product needs a commercial license from the author or route B.
  `libcynipsnet.a` is ST SLA0104.

D. Use the NPU budget better

- Compute is the constraint, not memory: the weights use 7 of the 107 MB NOR budget. Today,
  2.58 GMAC takes 220 ms, about 12 GMAC/s against a 600 GOPS peak. The run is memory-

bound, with
weights streamed from NOR on every window.

- The 12× redundancy is the biggest waste. Options: train for a shorter window, increase the
  stride (1 s halves the compute but adds latency), or move to a chunked or streaming design.
  The compute freed can pay for a larger model; the QuartzNet paper reports 10x5 at 4.14 / 12.33 dev WER versus 5x5 at 5.39 / 15.69.
- C2's dilation is materialized as 173 taps, half of them zeros. This is a small cost.
- The log reports 0 epoch-controller blobs. Test whether ST Edge AI's epoch controller and its
  memory-placement options reduce the 268-epoch CPU overhead.

6. Accuracy expectations

"Perfect" is not reachable for any ASR model. Define the target on our own test set: our speakers,
our phrases, the board microphone at 0.5, 1 and 2.5 m, measured as WER and command accuracy.
Published baselines for the shipped model, with greedy decoding:

Set	WER
test-clean	5.89 %
test-other	16.06 %
dev-other at 50 cm	17.04 %
dev-other at 250 cm	26.02 %

The distance and accent results are where the largest gains are.